Correction Request: Faults in the Released Implementation of "Multimodal Prompt Learning with Missing Modalities for Sentiment Analysis and Emotion Recognition" (ACL 2024)

Ashish Kumar Dash, Sujal Som and Tanmay Jaiswal
Indian Institute of Technology Bhilai, India
{ashishkd, sujals, tanmay}@iitbhilai.ac.in

***Abstract*—To Zirun Guo, Tao Jin and Zhou Zhao (Zhejiang University; Shanghai Artificial Intelligence Laboratory). We audited the public code release https://github.com/zrguo/MPLMM (commit `c6f8d18`) on CMU-MOSI as part of a course project at IIT Bhilai. Your README notes that the repository is a reconstructed code base and invites error reports, so we report eight faults, each located at a line number, with the measured effect and a proposed fix. The most consequential are as follows. (1) The missing-type prompt and its three coupling matrices are zero-initialised on both sides of a bilinear product, so they receive exactly zero gradient and stay zero in every run; the released code therefore cannot produce the missing-type ablation of Table 3. A re-initialisation to $\mathcal{N}(0, 0.02)$ makes all four tensors train. (2) Final metrics are computed on the validation split, not the test split. (3) Modality dropout is applied to the validation and test splits. (4) The default schedule stops before convergence. With the released defaults we obtain 57.55% six-case Acc-2 on MOSI against the reported 72.14%; a corrected schedule gives 67.51%. We do not claim that the paper's reported numbers are wrong. We ask for the fixes below, or for the configuration that produced the paper's tables.**

***Index Terms*—Reproducibility, code audit, missing modalities, prompt learning, CMU-MOSI.**

I. Setup

- **Code:** upstream `c6f8d18`, kept unmodified as a submodule. Compatibility shims (items 8b and 8c below) are applied at run time by a wrapper.
- **Hardware and software:** one NVIDIA RTX 3060 (12 GB), PyTorch 2.14.
- **Data:** the MulT CMU-MOSI feature release (GloVe-300, COVAREP-5, Facet-20), with splits 1,284 / 229 / 686.
- **Backbone:** MulT pretrained on CMU-MOSEI, then frozen.
- **Metric:** the *six-case forced mean Acc-2*. Each of the six missing-modality cases is forced on all 686 test samples, and the six accuracies are averaged.
- **Precision:** the binomial precision floor at $n = 686$ is 1.91 points.

II. Faults and Proposed Corrections

Table I summarises the eight items. Line numbers refer to `c6f8d18`.

A. The missing-type prompt never trains (item 1)

Observation. In `PromptModel.__init__`, `missing_type_prompt` $(3, \ell_p, d_p)$ and `m_a`, `m_v`, `m_l` are created with `torch.zeros`. The coupling $\mathbf{m}_p$ is built from the missing-signal prompts times `m_*` (lines 340–380). The prompt actually used is `batch_prompt = missing_type_prompt @ mp` (lines 410–411). In a bilinear product, each factor's gradient is proportional to the other factor, so if both start at zero they stay exactly zero under any optimiser.

Evidence. Five independent checks agree:

- hand derivation;
- a standalone reimplementation, in which both norms stay 0 for 200 steps;
- a state-dict check: the four tensors are absent from the backbone checkpoint, so weight transfer cannot seed them;
- functional perturbation with $\mathcal{N}(0, 100)$ noise, giving maximum output change 0;
- Adam at learning rate 1 with weight decay.

The tensors are all-zero in 55 of 55 trained runs that record the flag, and on real trained weights. In every forward pass, 3×16 inert tokens are prepended to the three transformers.

Impact. The missing-type component of the Table 3 ablation cannot be reproduced from this code. By Table 3 itself the component is worth about 0.2 points (71.94 vs 72.14), so this fault does *not* explain the reproduction gap.

Proposed patch (executed). Re-initialise only these four tensors:

```
- torch.zeros(3, self.prompt_length, self.prompt_dim)
+ 0.02 * torch.randn(3, self.prompt_length,
+                     self.prompt_dim)
- self.m_a = nn.Parameter(torch.zeros(self.alen,
-                                      2 * self.prompt_dim))
+ self.m_a = nn.Parameter(0.02 * torch.randn(self.alen,
+                                      2 * self.prompt_dim))
  (likewise m_v, m_l)
```

We compared both initialisations from the same seed over 50 Adam steps on all seven missing cases. With the released initialisation, every gradient, every displacement, $\mathbf{m}_p$ and the prompt's effect on the output remain exactly 0. With the patch, gradients become non-zero from step 2, once the missing-signal prompts leave zero. All four tensors then move

TABLE I
SUMMARY OF LOCATED FAULTS (UPSTREAM COMMIT c6f8d18).

#	location	fault	measured effect	proposed fix
1	`src/model.py:225--230`	missing-type prompt and couplings `m_a`, `m_v`, `m_l` zero-initialised; bilinear product	zero gradient; all-zero in 55/55 trained runs; Table 3 missing-type ablation not reproducible	$\mathcal{N}(0, 0.02)$ initialisation (verified to train)
2	`src/train.py:160` (and 140)	final metrics computed with `test=False`	reported numbers are 229 validation samples	`test=True`
3	`src/utils.py get_data; src/mosidata.py:55--56`	`drop_rate` applied to valid and test	stochastic evaluation and selection signal	dropout on train only; force each case at test
4	`src/train.py:144, 154`	selection and LR decay on validation loss; paper reports accuracy	contributes to the gap with 2, 3 and 5	document, or select on validation accuracy
5	`main.py:78--80`	defaults: 40 epochs, `when`=10, `drop_rate` 0.6	57.55% vs 72.14% reported; corrected schedule 67.51%	publish the exact configuration per table
6	`src/model.py:187--201`	nine time-mixers hold $\ell_{out}(\ell_p + \sum \ell_{in}) + \ell_{out}$ weights each	1,115,666 trainable at unaligned length, 104% of the backbone	disclose the length scaling
7	`src/model.py:469--472`	`MLPLayer is_Fusion` branches are identical	flag has no effect	remove, or restore the intended operator
8	README; `utils.py:81; train.py:27, 66, 119, 159`	undeclared dependencies; `torch.load`/scheduler incompatible with current PyTorch; unconditional `DataParallel`	clean install fails; own checkpoints do not load	requirements file; patches below

(maximum displacement 0.022–0.029), $\max |\mathbf{m}_p| = 0.0101$, and zeroing the prompt changes the output by up to 0.953. We have verified the mechanism, not its accuracy.

B. Evaluation on the validation split (item 2)

`evaluate(model, criterion, test=False)` at `train.py:160` selects `valid_loader` (line 97). The test loss is computed at line 140 but its predictions are discarded. Every printed metric is therefore on the 229-sample validation split.

```
- _, results, truths = evaluate(model, criterion,
-                               test=False)
+ _, results, truths = evaluate(model, criterion,
+                               test=True)
```

C. Modality dropout on evaluation splits (item 3)

`get_data` passes `drop_rate=args.drop_rate` for every split. `MOSIData.__getitem__` then draws a random missing case for each sample (`mosidata.py:55--56`). Validation loss, checkpoint selection, the LR scheduler and the reported metrics are therefore computed on a randomly corrupted, seed-dependent mixture rather than a fixed protocol. We suggest applying dropout to training only, and reporting each missing case by forcing it on the full test set, as in the paper's per-case tables.

D. Selection on loss, and default schedule (items 4 and 5)

Checkpoints are saved when validation loss improves (`train.py:154`), and `ReduceLROnPlateau` steps on the same loss (line 144). The paper reports accuracy. This is a common choice; we list it because it interacts with items 2 and 3.

The defaults in `main.py:78--80` are 40 epochs, decay patience 10 and `drop_rate` 0.6, whereas the paper's reported optimum is 0.7. In our runs this schedule stops before convergence and then decays the learning rate 100×.

E. Undisclosed length scaling and a dead flag (items 6 and 7)

Each generation layer flagged `is_Fusion=True` is a `Conv1d(kernel_size=1)` applied after a transpose. The convolution's channel axis is therefore *time*, so the layer is a dense, position-indexed map over time and pointwise over features. Perturbing one input time step moves 50 of 50 output positions; perturbing one feature moves 1 of 30. Its parameter count, $\ell_{out}(\ell_p + \sum \ell_{in}) + \ell_{out}$, is verified by instantiation. Total trainable parameters are 88,141 at aligned length (8.2% of the 1,071,211-parameter backbone) and 1,115,666 at unaligned length (375/500), which is 104%. The paper's 5–10% figure holds only for aligned inputs. Separately, the two branches of `MLPLayer.__init__` selected by `is_Fusion` (`model.py:469--472`) are identical, so the flag has no effect. If a different operator was intended for the fusion layers, it may have been lost in the reconstruction.

F. Environment (item 8)

a) The README lists only Python ≥3.8 and PyTorch ≥1.8. `src/utils.py` imports `src/iemodata.py`, which requires `h5py`, `torchvision` and PIL even for MOSI.
b) Checkpoints are pickled whole models. Since PyTorch 2.6, `torch.load` defaults to `weights_only=True`, so `utils.py:81` and `train.py:159` need `weights_only=False`.
c) The `verbose` argument of `ReduceLROnPlateau` (`train.py:27`) raises `TypeError` on current PyTorch.
d) `nn.DataParallel` is applied whenever `batch_size`>10 (`train.py:66, 119`), regardless of GPU count.

```
- model = torch.load(pretrained)
+ model = torch.load(pretrained, weights_only=False)
- model = torch.load(hyp_params.name)
+ model = torch.load(hyp_params.name, weights_only=False)
- optimizer, mode="min", patience=hyp_params.when,
-     factor=0.1, verbose=True)
```

```
+ optimizer, mode="min", patience=hyp_params.when,
+     factor=0.1)
```

Patches (b) and (c) are the runtime shims we used for every run. The patch for item 2 is a one-argument change. The item-3 suggestion is not a tested patch.

III. Reproduction Results

TABLE II
CMU-MOSI six-case forced mean Acc-2 (%), upstream training loop, aligned features, seed 1.

configuration	Acc-2
released defaults	57.55
corrected schedule (batch 64, 200 epochs, `when` 30)	67.51
paper (reported)	72.14

Against the paper, the corrected run's deficit is -6.45 points when text is absent (cases 0, 3, 4) and -2.82 when text is present (cases 1, 2, 5). The backbone itself is healthy: a MulT trained from scratch reaches 80.18% on complete data, and freezing it costs 2.1 points. The corrected baseline is one seed, and the remaining 4.63-point gap is unexplained. We have not reproduced the paper's stated 30-epoch setting.

IV. Requests

1) Confirm the intended initialisation of the missing-type prompt and couplings, and whether Table 3 was produced with a different code version.
2) Apply, or advise on, the fixes for items 2, 3 and 8.
3) Publish the exact command lines, seeds and configuration for the main MOSI results and Table 3, including the epoch count, `when`, `drop_rate` and checkpoint-selection criterion.
4) State the generation module's parameter count as a function of sequence length, or restrict the 5–10% figure to aligned inputs.
5) Clarify the intended operator behind `is_Fusion`.

We are glad to open these as GitHub issues and to submit the patches as a pull request.

V. For Information: Results Beyond the Faults

These results come from our own training loop and are not comparable with Table II.

- Replacing the nine dense time-mixers with a length-invariant cross-attention operator matches accuracy at $\ell = 50$, 100, 200 and native length. At native length the two are equivalent within ± 2 points (TOST $p = 0.033$, two machines), with $5.86\times$ fewer parameters.
- Replacing the time-mixers with a zero-parameter average pool costs no detectable accuracy at $\ell = 200$.
- The logged generated-vs-real CKA mostly measures padding position: MOSI audio and vision are 86–90% padding.
- Swapping Facet for CLIP ViT-B/32 features extracted from the raw MOSI clips changes accuracy by no detectable amount.

Acknowledgment

We thank the authors for releasing their code. This note was prepared as part of the CS550 Machine Learning course project at IIT Bhilai. Audit tools and run records are in the project repository, https://github.com/Ashish-Kumar-Dash/ML-Project.

References

[1] Z. Guo, T. Jin, and Z. Zhao, "Multimodal prompt learning with missing modalities for sentiment analysis and emotion recognition," in *Proc. 62nd Annu. Meeting Assoc. Comput. Linguistics (ACL)*, 2024, pp. 1726–1736.
[2] Y.-H. H. Tsai *et al.*, "Multimodal transformer for unaligned multimodal language sequences," in *Proc. ACL*, 2019, pp. 6558–6569.
[3] A. Zadeh, R. Zellers, E. Pincus, and L.-P. Morency, "MOSI: Multimodal corpus of sentiment intensity and subjectivity analysis in online opinion videos," arXiv:1606.06259, 2016.